Software engineering practices and AI tools for research code

William S Hart

Infectious Disease Modelling Group
Mathematical Institute, University of Oxford

Group meeting, 29 October 2026



Oxford Mathematics

University of Oxford
Mathematical Institute

This talk

Check a result, rebuild it reliably, and delegate coding while retaining scientific control.

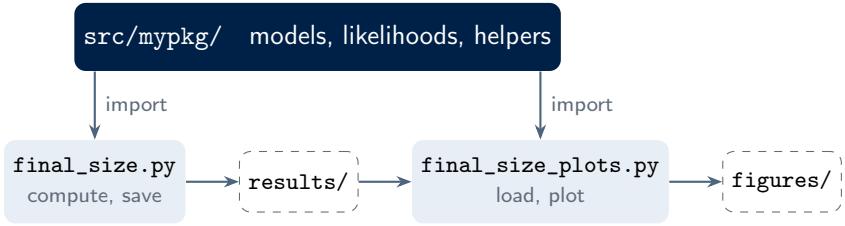
- **Part 1:** practices and tools for reliable, reproducible code.
- **Part 2:** AI coding tools: uses, good habits and risks.

> **Disclaimer:** these slides were prepared with AI assistance and may contain errors. Check advice against the documentation and your project's needs.

1. Practices and tooling

2. AI tools

Organising code



Example: vary R_0, compute final sizes, then plot the curve.

- **Don't repeat yourself:** put repeated logic in functions; define parameters once.
- **One function per task:** `compute_final_size`, `plot_final_size`. Pass inputs explicitly.
- **Public vs private:** keep a small, stable API. In Python, use `_helper` for private helpers.

Nine practices: my adoption priorities

Mathematical Institute

Version control ■■■■■

Environments ■■■■■

Formal testing ■■■■□

Editor and debugger ■■■■□

Formatting and linting ■■■□□

Workflow management ■■■□□

Type checking ■■□□□

Continuous integration ■■□□□

Archiving code ■■□□□

■■■■■ Priority for a typical analysis repo (5 = start here)

my choice | other tools — Tools for Python, R and Julia

Version control

University of Oxford
Mathematical Institute

Record changes; restore committed versions.

Priority

- See what changed, when and why.
- Push to a remote for backup and collaboration.
- Branches let features develop in parallel.
- Commit before an agent edits, then review its diff.

Tools

Any	git
Hosting	GitHub, GitLab
Interfaces	VS Code, GitHub Desktop, GitHub CLI (gh)

Environments and package management

University of Oxford
Mathematical Institute

A recorded environment for each project.

Priority

- Code depends on package versions (e.g. numpy 2).
- A shared global environment drifts and breaks old projects.
- Commit an environment specification with package versions.
- A lockfile also records resolved dependencies, making rebuilding easier.

Tools

Python	**pixi** · uv · conda/mamba · pip + venv
R	**renv** · pak · rig
Julia	Pkg
Containers	Docker · Apptainer

Formal testing

University of Oxford
Mathematical Institute

Automate checks; catch regressions.

Priority ■■■■□

- **Basic checks are essential.** The score is for a formal test suite.
- Use analytic answers, limiting cases, invariants and numerical convergence.
- Pass a seeded `rng` for repeatability; check distributions separately.
- Test-driven development (TDD): write a failing test, then implement.

Tools

Python	**pytest** hypothesis pytest-cov
R	**testthat**
Julia	Test

Editor and debugger



See errors as you type; inspect, don't print.

Priority ■■■■□

- The editor flags errors and makes code easy to navigate.
- A debugger pauses the code so you can inspect its state.
- Scripts or notebooks: make dependencies explicit and rerun from a clean state.

Tools

Editors	**VS Code**	PyCharm	Positron	RStudio
Debuggers	**VS Code debugger**	pdb	R browser()	Infiltrator.jl

Formatting and linting

University of Oxford
Mathematical Institute

Consistent style; common mistakes flagged.

Priority

- **Formatters** standardise spacing and line breaks.
- **Linters** flag likely bugs and bad patterns without running the code, e.g. undefined names, unused imports.
- Both run on save, or after each agent edit (a hook: a command run automatically).

Tools

Python	**ruff** pylint
R	air styler lintr
Julia	JuliaFormatter.jl Runic.jl
Any	pre-commit

Workflow management

University of Oxford
Mathematical Institute

One command rebuilds every result.

Priority ■■■□□

- Reruns affected steps when declared inputs change.
- Makes the dependencies between data, code and outputs explicit.
- Preserve raw inputs; record sources and versions; validate data on loading.
- Scales to the cluster; matters more as projects grow.

Tools

Any	**Snakemake** Nextflow Make
R	targets
Simple	pixi tasks

Type checking

University of Oxford
Mathematical Institute

Catch wrong-type bugs before running.

Priority ■■□□□

- E.g. passing `None` where an array is expected.
- Type hints document interfaces, and help editors and AI.
- Useful for stable interfaces; AI can help add hints.

Tools

Python	**ty (beta)** · pyrefly · mypy · pyright
Julia	JET.jl
R	none mainstream

Continuous integration

University of Oxford
Mathematical Institute

Tests run on every push, on clean machines.

Priority ■■□□□

- Catches “works on my machine” and broken lockfiles.
- Can test several operating systems.
- Analysis repos can run a small smoke test; shared packages need fuller checks.

Tools

Pipelines	**GitHub Actions**	GitLab CI
Coverage	**Codecov**	

Packages other people install: ■■■■□

Archiving code

University of Oxford
Mathematical Institute

A permanent copy of the code behind each paper.

Priority ■■□□□

- A one-off step at publication: archive the exact version used for the paper.
- A DOI keeps the paper's link permanent, even if the repo moves.
- Zenodo's GitHub integration needs a public repo; otherwise upload the release by hand.

Tools

Archives: **Zenodo** | Software Heritage | OSF

At publication: ■■■■□

What goes in a repo

```
project/
├── README.md   what it is, install, reproduce
├── LICENSE   no licence = no reuse
├── .gitignore
├── pyproject.toml*   environment spec
├── pixi.lock*   exact versions
├── AGENTS.md*   project instructions
├── src/mypkg/*   reusable code as a package
├── scripts/*   analysis + plot script pairs
│   ├── final_size.py   compute, save results
│   └── final_size_plots.py   load results, plot
├── tests/
├── data/   inputs and provenance; no sensitive data
├── results/   outputs (committed: my choice)
├── figures/
└── Snakefile*   reproduction workflow
```

Essential

Good practice

Colour rates the practice.

* My choice of format, tool or layout; alternatives work too.

Documenting code

- README: what it is, how to install it, how to reproduce the results.
- Docstrings (roxygen comments in R): purpose, arguments, units, returns.
- Comments say *why*, not *what*.
- Names say what things are (`incubation_period`, not `ip`).
- Remove obsolete code once its history is committed.
- Doc websites are useful for shared packages: **Sphinx** | roxygen2 + pkgdown | Documenter.jl

Additional resources

- *Best practices for scientific computing*:
  Wilson et al. (2014), *PLoS Biol.*, https://doi.org/10.1371/journal.pbio.1001745
- *Good enough practices in scientific computing*:
  Wilson et al. (2017), *PLoS Comput. Biol.*, https://doi.org/10.1371/journal.pcbi.1005510
- *The Turing Way*, a handbook for reproducible research:
  `https://book.the-turing-way.org`
- Oxford Research Software Engineering group training:
  `https://www.rse.ox.ac.uk/training`

1. Practices and tooling

2. AI tools

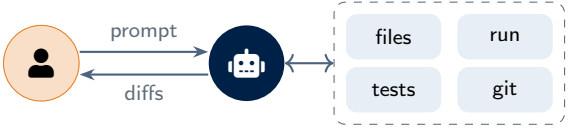
Options: chat assistants and coding agents

Chat
ChatGPT, claude.ai



You are the loop.
Good for questions, snippets.

Coding agent
Claude Code,
Codex



It runs the loop in your repo; you review.

Claude Code terminal, VS Code, desktop, web **OpenAI Codex** CLI, IDE, cloud
Also: GitHub Copilot (agent mode), Cursor, Gemini CLI.

How much to delegate



Autocomplete, explanations, reviewing your code. More hands-on practice.

One function or feature at a time, reading every diff.

Accepting code without reading it. Throwaway exploration; check any result you use.

Plan the science before the code

Mathematical Institute

You decide, before any code

- The model, its assumptions and the data.
- Parameter values and priors, and their sources.
- Fitting, diagnostics and checks of model adequacy.
- What correct output looks like: known answers and limiting cases (your tests).

Talk it through with the agent if you like; the decisions are yours.

AI can help with the maths

- Deriving results, e.g. the final size or R_0 of your model.
- Checking your derivations for slips.
- Finding limiting cases and known answers to test against.
- Checking the code matches the equations.

✔ Verify independently: by hand, with SymPy, or numerically.

Uses: checking and testing

- **Bugs:** ask it to review code, or give it the traceback; ask for the root cause, not a patch.
- **Tests:** AI can help write unit tests, suggest edge cases and extend coverage.

Example: final size (fraction ever infected) in a closed SIR model

A weak check

Compare the result with a second call to the same function.

The same bug can appear on both sides.

An independent check

No transmission, $R_0 = 0$:

final size = initial infected fraction.

Uses: grunt work and more

Mathematical Institute

Grunt work

Tedious for you, quick for you to check.

- Type hints and docstrings
- Renames across files
- New-but-similar sensitivity analyses
- Turning copy-paste into functions
- Updating for new library versions

And more

Reading, translating, explaining.

- Understanding unfamiliar code
- Translating R or MATLAB to Python
- Decoding install and HPC errors
- READMEs, commits, merge conflicts
- Slides for a supervisor meeting

Prompting

- **Be specific:** the files involved, a pattern to follow, what "fixed" looks like.
- **Let it check its work:** tests or a known result. Ask for evidence.
- **Bigger tasks:** plan first, or ask it to interview you.
- **Exploring:** vague is fine ("what would you improve here?").

Vague

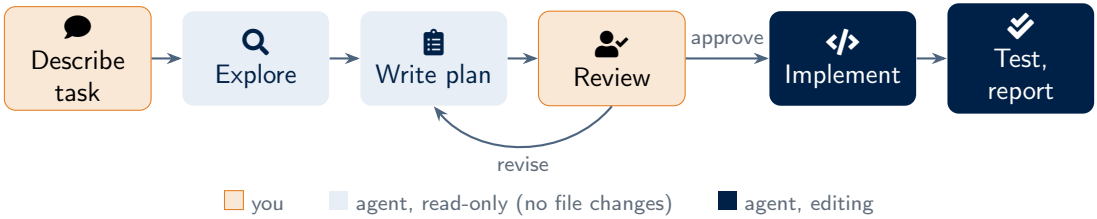
"fix the bug in the plot"

Specific

"the final-size plot has a percentage axis but shows fractions. Check `scripts/final_size_plots.py`; verify that 0.2 displays as 20%, fix the units, and show me the corrected figure"

Docs: code.claude.com/docs/en/best-practices

Planning with an agent



✔ Use it

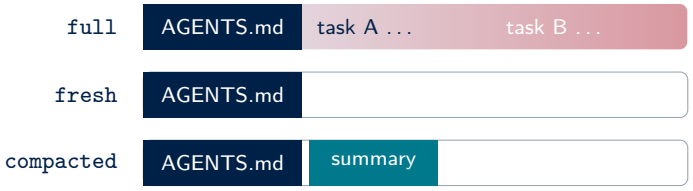
When you're unsure of the approach, or several files change: use plan mode, or ask for a plan and no edits.

✖ Skip it

For small, well-understood edits.

Docs: code.claude.com/docs/en/permission-modes

Sessions and context



Long, mixed context can reduce quality.

New session between unrelated tasks.

Summarise between linked tasks, saying what to keep.

- Two failed corrections? Start afresh with a better prompt.
- Interrupt early. Checkpoints restore captured edits; git restores committed files. Neither reverses every command.
- Review in a fresh session or with a second model to reduce anchoring.

Docs: code.claude.com/docs/en/best-practices

Agent instruction files

Mathematical Institute

Project
Committed (`AGENTS.md`): the project's conventions, for everyone.

Personal, this project
Your preferences; not committed.

Personal, all projects
In your home config.

- Loaded every session: commands and conventions the agent can't infer.
- Keep it short: would removing this line cause mistakes?
- Add a rule when you find yourself repeating a correction.
- `AGENTS.md` is an open format read by many coding agents.

Docs: code.claude.com/docs/en/memory

Skills

- Instructions plus templates and scripts, loaded when relevant or on request.
- Load detailed procedures when needed.
- **Make one** "when you keep pasting the same instructions, checklist, or multi-step procedure into chat" (within or across projects).
- Examples: setting up a new project repo; making slides in a group template; a checklist for adding an analysis.
- An open standard (agentskills.io), usable by other tools.

Docs: code.claude.com/docs/en/skills

Other useful features (reference)

Mathematical Institute

Feature	Use
Hooks	Commands run automatically at set points, e.g. format and lint after every edit (as in this repo).
Checkpoints	Rewind the agent's file edits and the conversation.
Permission modes	Choose what it may do without asking, and know which mode you're in.
Sandbox	Limits the files and network its commands can reach.
Fresh-context review	A second session or model reviews the diff, with less anchoring to the original implementation.
Subagents	Helper agents with their own fresh context, for side investigations.
Parallel sessions	Separate checkouts (git worktrees), so several sessions can work at once.
Connectors (MCP)	External tools and data, e.g. PubMed, GitHub.

Reference: names link to Claude Code docs; most agents have equivalents.

Risks: correctness and skills

Mathematical Institute

Plausible but wrong

A plausible curve from a gamma scale/rate mix-up.

✔ Check against analytic results, limiting cases, simulate-and-recover.

Tests that pass for the wrong reason

Tests mirror the code, or are edited or skipped to pass.

✔ Decide what the tests check, and read them.

Bloat and hidden fallbacks

More code than needed; fallbacks that hide errors.

✔ Small steps, read every diff, ask for simpler code.

Losing understanding

You can't defend code you don't understand.

✔ Ask it to explain; write some code yourself.

Risks: data, security and policy

Mathematical Institute

Confidential or patient-level data

Hosted tools send content and tool output to the provider.

✔ Use Oxford ChatGPT Edu/Codex via SSO. Check ethics and data agreements; develop on synthetic data.

Destructive commands

Agents may run commands without asking.

✔ Use suitable permissions and a sandbox; commit first.

Untrusted packages and instructions

Malicious dependencies or instructions in files and web pages.

✔ Check dependency names and sources before installing; limit permissions.

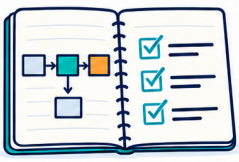
Journal, funder and thesis rules

AI can't be an author; policies vary.

✔ Check them, and say how AI was used.

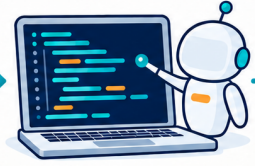
You own every line, whoever wrote it

1 Plan



Model, assumptions and units
Choose independent checks

2 Implement



Small, reviewable changes
Read every diff

3 Validate



Known answers and limiting cases
Check numerical accuracy

4 Reproduce



Record data and parameters
Rebuild and archive results

FOUNDATIONS



Version control



Recorded environment



Automated checks



README +
agent instructions